

Week 5 Deliverables

Traffic Congestion Prediction — AutoResearch
STAT 390 Capstone | Sherry Huang | 2026

Best Result (exp_020): F1 = 0.6727 | Precision = 0.6441 | Recall = 0.7039

Improvement vs baseline: +0.1129 F1 (+20.2%) | +0.2591 recall (+58.2%) | 21 experiments, 0 crashes

1 Complete Experiment Log Bundle

All 21 experiments from Weeks 3–5. **Green row** = current best (exp_020). **Blue row** = baseline (exp_001).
Status column: keep (green), discard (red), baseline (blue).

ID	Description	F1	Prec	Recall	Status
exp_001	Baseline logistic regression lags+time	0.5598	0.7549	0.4448	baseline
exp_002	Logistic regression class_weight=balanced	0.6363	0.5325	0.7903	keep
exp_003	Random forest n=100 depth=8 class_weight=balanced	0.6450	0.5631	0.7547	keep
exp_004	RF balanced extended features lags1-6 rolling_mean speed_diff	0.6523	0.5735	0.7562	keep
exp_005	HistGradientBoosting max_iter=200 depth=6 lr=0.05 balanced	0.6471	0.5468	0.7925	discard
exp_006	RF n_estimators=50 depth=8 balanced extended	0.6494	0.5695	0.7554	keep
exp_007	RF n_estimators=200 depth=8 balanced extended	0.6566	0.5781	0.7598	keep
exp_008	RF n_estimators=100 depth=4 balanced extended	0.6524	0.5692	0.7642	keep
exp_009	RF n_estimators=100 depth=12 balanced extended	0.6526	0.6127	0.6981	keep
exp_010	RF balanced depth=8 n=100 minimal features	0.6450	0.5631	0.7547	keep
exp_011	RF balanced depth=8 n=100 all features incl rolling_std MONTH	0.6552	0.5771	0.7576	keep
exp_012	RF n=100 depth=8 no class_weight extended features	0.6235	0.7697	0.5239	discard
exp_013	Downsampling 2:1 threshold=0.5 RF n=200 depth=8	0.6585	0.6538	0.6633	keep
exp_014	Downsampling 1:1 threshold=0.5 RF n=200 depth=8	0.6520	0.5581	0.7837	keep
exp_015	threshold=0.40 class_weight=balanced RF n=200 depth=8	0.6408	0.5211	0.8316	keep
exp_016	threshold=0.35 class_weight=balanced RF n=200 depth=8	0.6316	0.4983	0.8621	keep
exp_017	Downsampling 2:1 + threshold=0.40 combined RF n=200 depth=8	0.6560	0.5885	0.7409	keep
exp_018	Max-recall downsampling 1:1 + threshold=0.35 RF n=200 depth=8	0.6235	0.4847	0.8737	keep

ID	Description	F1	Prec	Recall	Status
exp_019	Segment-relative features + undersample 2:1 + threshold=0.50	0.6654	0.7202	0.6183	keep
exp_020	Segment-relative features + undersample 2:1 + threshold=0.40	0.6727	0.6441	0.7039	keep
exp_021	Segment features + class_weight=balanced + threshold=0.40	0.6641	0.5600	0.8157	keep

Resource notes: All 21 runs completed in 2.7–6.7 s on CPU (no crashes). Python 3.9.6 / scikit-learn 1.6.1 / macOS CPU only. Git history: github.com/SherryHuang0311/traffic-autoresearch

2 Metric Trajectory

The metric trajectory plot is saved as **metric_trajectory.png** in this same deliverables/week5/ folder. It contains three panels:

Panel 1: F1 score over all 21 experiments with the best-so-far envelope highlighted in green

Panel 2: Precision and recall tracked separately across all runs

Panel 3: Bar chart of F1 by experiment, coloured by status (green=keep, red=discard, blue=baseline), with week-band shading

Key trajectory observations:

Weeks 3–4	F1 rose from 0.5598 → 0.6566 through model selection (LR → RF) and feature engineering (lags 1–6, rolling mean, speed_diff).
Week 5 early	Downsampling and threshold tuning explored the precision/recall trade-off (threshold 0.35–0.50) but could not raise the F1 ceiling beyond 0.6585.
Week 5 late	Segment-relative features (speed_zscore, speed_vs_seg_mean, segment_mean_speed) broke the ceiling — F1 jumped to 0.6727, a gain of +0.0142 in a single step.

3 Keep / Discard / Crash Summary

Outcome	Count	Details
Keep	19	All 19 kept experiments improved upon or explored meaningfully from the baseline. Best: exp_020, F1=0.6727.
Discard	2	exp_005: HistGradientBoosting scored F1=0.6471, below the then-current best of 0.6523. model.py reverted. exp_012: Removing class_weight dropped F1 to 0.6235 and recall to 0.524. Confirmed class balancing is essential. Reverted.
Crash	0	No crashes across all 21 experiments. One pre-experiment ModuleNotFoundError (sklearn not installed) occurred during environment setup and was not logged as an experiment.

Rollback discipline: Every discard had model.py reverted to the previous best configuration before the next experiment ran. The results.csv log reflects the true status of each run with no post-hoc editing.

4 Best Result vs. Baseline

Metric	Baseline — exp_001 Logistic Regression, lags+time	Best — exp_020 RF undersample 2:1, seg. features, threshold=0.40	Change
Validation F1	0.5598	0.6727	+0.1129 (+20.2%)
Precision	0.7549	0.6441	−0.1108
Recall	0.4448	0.7039	+0.2591 (+58.2%)
Runtime	3.29 s	2.99 s	−0.30 s

What drove the improvement (in order of impact):

1	class_weight=balanced (Week 3) Fixed the biggest recall gap instantly — the model had been ignoring 77% of congestion events.
2	Random Forest (Week 3) Better at nonlinear speed patterns than logistic regression; F1 up ~1.5 points.
3	Extended lag features lag_1–6 + rolling_mean + speed_diff (Weeks 3–4) More speed history gives temporal context; F1 up ~0.8 points.
4	Undersampling majority 2:1 (Week 5) Cleaner training signal than weight adjustment alone; pushed precision higher while keeping recall.
5	Segment-relative features: speed_zscore, speed_vs_seg_mean, segment_mean_speed (Week 5) Broke the global threshold ceiling by giving the model road-specific context; +0.0142 F1 in one step.

5 What Actually Worked — Memo

The single most impactful discovery of the project was identifying why the model was stuck at $F1=0.658$ and fixing it.

The root cause

The congestion label was defined globally: any segment with speed below 23 mph (the 30th percentile of all training speeds) is labeled congested. This sounds reasonable, but Chicago has roads with very different normal speeds — highways averaging 45 mph and arterials averaging 15 mph share the same cutoff. About **16.3% of training rows were mislabeled** as a result. A highway running at its normal slow-peak pace was called congested. A side street moving unusually fast was called not congested. **No model can learn correctly from wrong labels.**

The fix

Four new features were added to the feature superset, computed from training data only (no leakage into validation or test):

Feature	Formula	What it captures
segment_mean_speed	mean(SPEED) per segment, train only	What is 'normal' for this road
segment_std_speed	std(SPEED) per segment, train only	How variable this road's speed is
speed_vs_seg_mean	$SPEED / \text{segment_mean_speed}$	How fast relative to this segment's norm
speed_zscore	$(SPEED - \text{mean}) / \text{std}$	How many std deviations from normal

Instead of asking *"is 22 mph slow?"* the model can now ask *"is 22 mph slow for this specific road?"* That is the right question.

What did NOT work

Hyperparameter tuning (Weeks 3–4) moved F1 by less than 0.5% across six experiments — algorithmic tuning cannot fix label noise. Threshold tuning (Week 5) successfully traded precision for recall but could not raise the F1 ceiling. HistGradientBoosting underperformed Random Forest on this dataset and was discarded.

What worked well structurally

The loop itself ran cleanly. All 21 experiments completed without a single crash. Every discard was caught, logged, and reverted correctly. The frozen run.py prevented evaluation drift — every experiment used the exact same data split, congestion threshold derivation, and F1 metric. Results are fully reproducible (random_state=42 throughout).

Remaining limitation

The global congestion threshold still defines the target label. Even with segment-relative features helping the model understand each road better, the label itself is still globally defined. A fully segment-relative label definition would require changing the frozen evaluation logic — a decision for future weeks.

Current best model (exp_020): Random Forest, 200 trees, max depth 8, trained on a 2:1 undersampled dataset, 14 features including speed_zscore, speed_vs_seg_mean, and segment_mean_speed, decision threshold 0.40.

F1 = 0.6727 | Precision = 0.6441 | Recall = 0.7039